



Published in final edited form as:

Adv Neural Inf Process Syst. 2019 December ; 32: 7834–7844.

Stein Variational Gradient Descent with Matrix-Valued Kernels

Dilin Wang*, Ziyang Tang*, Chandrajit Bajaj, Qiang Liu

Department of Computer Science, UT Austin

Abstract

Stein variational gradient descent (SVGD) is a particle-based inference algorithm that leverages gradient information for efficient approximate inference. In this work, we enhance SVGD by leveraging preconditioning matrices, such as the Hessian and Fisher information matrix, to incorporate geometric information into SVGD updates. We achieve this by presenting a generalization of SVGD that replaces the *scalar-valued* kernels in vanilla SVGD with more general *matrix-valued* kernels. This yields a significant extension of SVGD, and more importantly, allows us to flexibly incorporate various preconditioning matrices to accelerate the exploration in the probability landscape. Empirical results show that our method outperforms vanilla SVGD and a variety of baseline approaches over a range of real-world Bayesian inference tasks.

1 Introduction

Approximate inference of intractable distributions is a central task in probabilistic learning and statistics. An efficient approximation inference algorithm must perform both efficient *optimization* to explore the high probability regions of the distributions of interest, and reliable *uncertainty quantification* for evaluating the variation of the given distributions. Stein variational gradient descent (SVGD) (Liu & Wang, 2016) is a deterministic sampling algorithm that achieves both desiderata by optimizing the samples using a procedure similar to gradient-based optimization, while achieving reliable uncertainty estimation using an interacting repulsive mechanism. SVGD has been shown to provide a fast and flexible alternative to traditional methods such as Markov chain Monte Carlo (MCMC) (e.g., Neal et al., 2011; Hoffman & Gelman, 2014) and parametric variational inference (VI) (e.g., Wainwright et al., 2008; Blei et al., 2017) in various challenging applications (e.g., Pu et al., 2017; Wang & Liu, 2016; Kim et al., 2018; Haarnoja et al., 2017).

On the other hand, standard SVGD only uses the first order gradient information, and can not leverage the advantage of the second order methods, such as Newton’s method and natural gradient, to achieve better performance on challenging problems with complex loss landscapes or domains. Unfortunately, due to the special form of SVGD, it is not straightforward to derive second order extensions of SVGD by simply extending similar ideas from optimization. While this problem has been recently considered (e.g., Detommaso et al., 2018; Liu & Zhu, 2018; Chen et al., 2019), the presented solutions either require

dilin@cs.utexas.edu.
*Equal contribution

heuristic approximations (Detommaso et al., 2018), or lead to complex algorithmic procedures that are difficult to implement in practice (Liu & Zhu, 2018).

Our solution to this problem is through a key generalization of SVGD that replaces the original scalar-valued positive definite kernels in SVGD with a class of more general *matrix-valued* positive definite kernels. Our generalization includes all previous variants of SVGD (e.g., Wang et al., 2018; Han & Liu, 2018) as special cases. More significantly, it allows us to easily incorporate various structured preconditioning matrices into SVGD updates, including both Hessian and Fisher information matrices, as part of the generalized *matrix-valued* positive definite kernels. We develop theoretical results that shed insight on optimal design of the matrix-valued kernels, and also propose simple and fast practical procedures. We empirically evaluate both Newton and Fisher based extensions of SVGD on various practical benchmarks, including Bayesian neural regression and sentence classification, on which our methods show significant improvement over vanilla SVGD and other baseline approaches.

Notation and Preliminary

For notation, we use bold lower-case letters (e.g., $\mathbf{x}$) for vectors in $\mathbb{R}^d$, and bold upper-case letters (e.g., $\mathbf{Q}$) for matrices. A symmetric function $k: \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ is called a positive definite kernel if $\sum_{i,j} c_i c_j k(\mathbf{x}_i, \mathbf{x}_j) \geq 0$ for any $\{c_i\} \subset \mathbb{R}$ and $\{\mathbf{x}_i\} \subset \mathbb{R}^d$. Every positive definite kernel $k(\mathbf{x}, \mathbf{x}')$ is associated with a *reproducing kernel Hilbert space* (RKHS) $\mathcal{H}_k$, which consists of the closure of functions of form

$$f(\mathbf{x}) = \sum_i c_i k(\mathbf{x}, \mathbf{x}_i), \quad \forall \{c_i\} \subset \mathbb{R}, \quad \{\mathbf{x}_i\} \subset \mathbb{R}^d, \quad (1)$$

for which the inner product and norm are defined by $\langle f, g \rangle_{\mathcal{H}_k} = \sum_{i,j} c_i s_j k(\mathbf{x}_i, \mathbf{x}_j)$,

$\|f\|_{\mathcal{H}_k}^2 = \sum_{i,j} c_i c_j k(\mathbf{x}_i, \mathbf{x}_j)$, where we assume $g(\mathbf{x}) = \sum_i s_i k(\mathbf{x}, \mathbf{x}_i)$. Denote by

$\mathcal{H}_k^d := \mathcal{H}_k \times \dots \times \mathcal{H}_k$ the vector-valued RKHS consisting of $\mathbb{R}^d$ vector-valued functions

$\boldsymbol{\phi} = [\phi^1, \dots, \phi^d]^\top$ with each $\phi^\ell \in \mathcal{H}_k$. See e.g., Berlinet & Thomas-Agnan (2011) for more rigorous treatment. For notation convenience, we do not distinguish distributions on $\mathbb{R}^d$ and and their density functions.

2 Stein Variational Gradient Descent (SVGD)

We introduce the basic derivation of Stein variational gradient descent (SVGD), which provides a foundation for our new generalization. See Liu & Wang (2016, 2018); Liu (2017) for more details.

Let $p(\mathbf{x})$ be a positive and continuously differentiable probability density function on $\mathbb{R}^d$.

Our goal is to find a set of points (a.k.a. particles) $\{\mathbf{x}_i\}_{i=1}^n \subset \mathbb{R}^d$ to approximate p , such that

the empirical distribution $q(\mathbf{x}) = \sum_i \delta(\mathbf{x} - \mathbf{x}_i)/n$ of the particles weakly converges to p when n is large. Here $\delta(\cdot)$ denotes the Dirac delta function.

SVGD achieves this by starting from a set of initial particles, and iteratively updating them with a deterministic transformation of form

$$\mathbf{x}_i \leftarrow \mathbf{x}_i + \epsilon \phi_k^*(\mathbf{x}_i), \quad \forall i = 1, \dots, n, \quad \phi_k^* = \operatorname{argmax}_{\phi \in \mathcal{B}_k} \left\{ -\frac{d}{d\epsilon} \operatorname{KL}(q_{[\epsilon\phi]} \| p) \Big|_{\epsilon=0} \right\}, \quad (2)$$

where ϵ is a small step size, $\phi_k^*: \mathbb{R}^d \rightarrow \mathbb{R}^d$ is an optimal transform function chosen to maximize the decreasing rate of the KL divergence between the distribution of particles and the target p , and $q_{[\epsilon\phi]}$ denotes the distribution of the updated particles $\mathbf{x}' = \mathbf{x} + \epsilon\phi(\mathbf{x})$ as $\mathbf{x} \sim q$, and $\mathcal{B}_k$ is the unit ball of RKHS $\mathcal{H}_k^d = \mathcal{H}_k \times \dots \times \mathcal{H}_k$ associated with a positive definite kernel $k(\mathbf{x}, \mathbf{x}')$, that is,

$$\mathcal{B}_k = \left\{ \phi \in \mathcal{H}_k^d: \|\phi\|_{\mathcal{H}_k^d} \leq 1 \right\}. \quad (3)$$

Liu & Wang (2016) showed that the objective in (2) can be expressed as a linear functional of ϕ ,

$$-\frac{d}{d\epsilon} \operatorname{KL}(q_{[\epsilon\phi]} \| p) \Big|_{\epsilon=0} = \mathbb{E}_{\mathbf{x} \sim q} [\mathcal{P}^\top \phi(\mathbf{x})], \quad \mathcal{P}^\top \phi(\mathbf{x}) = \nabla_{\mathbf{x}} \log p(\mathbf{x})^\top \phi(\mathbf{x}) + \nabla_{\mathbf{x}}^\top \phi(\mathbf{x}), \quad (4)$$

where $\mathcal{P}$ is a differential operator called *Stein operator*; here we formally view $\mathcal{P}$ and the derivative operator $\nabla_{\mathbf{x}}$ as $\mathbb{R}^d$ column vectors, hence $\mathcal{P}^\top \phi$ and $\nabla_{\mathbf{x}}^\top \phi$ are viewed as inner products, e.g., $\nabla_{\mathbf{x}}^\top \phi = \sum_{\ell=1}^d \nabla_{x^\ell} \phi^\ell$, with x^ℓ and ϕ^ℓ being the ℓ -th coordinate of vector $\mathbf{x}$ and ϕ respectively. With (4), it is shown in Liu & Wang (2016) that the solution of (2) is

$$\phi_k^*(\cdot) \propto \mathbb{E}_{\mathbf{x} \sim q} [\mathcal{P}k(\mathbf{x}, \cdot)] = \mathbb{E}_{\mathbf{x} \sim q} [\nabla_{\mathbf{x}} \log p(\mathbf{x}) k(\mathbf{x}, \cdot) + \nabla_{\mathbf{x}} k(\mathbf{x}, \cdot)]. \quad (5)$$

Such ϕ_k^* provides the best update direction for the particles within RKHS $\mathcal{H}_k^d$. By taking q to be the empirical measure of the particles, i.e., $q(\mathbf{x}) = \sum_{i=1}^n \delta(\mathbf{x} - \mathbf{x}_i)/n$ and repeatedly applying this update on the particles, we obtain the SVGD algorithm using equations (2) and (5).

3 SVGD with Matrix-valued Kernels

Our goal is to extend SVGD to allow efficient incorporation of precondition information for better optimization. We achieve this by providing a generalization of SVGD that leverages more general *matrix-valued* kernels, to flexibly incorporate preconditioning information.

The key idea is to observe that the standard SVGD searches for the optimal ϕ in RKHS $\mathcal{H}_k^d = \mathcal{H}_k \times \cdots \times \mathcal{H}_k$, a product of d copies of RKHS of scalar-valued functions, which does not allow us to encode potential correlations between different coordinates of ϕ . This limitation can be addressed by replacing $\mathcal{H}_k^d$ with a more general RKHS of vector-valued functions (called *vector-valued RKHS*), which uses more flexible *matrix-valued* positive definite kernels to specify rich correlation structures between different coordinates. In this section, we first introduce the background of vector-valued RKHS with matrix-valued kernels in Section 3.1, and then propose and discuss our generalization of SVGD using matrix-valued kernels in Section 3.2-3.3.

3.1 Vector-Valued RKHS with Matrix-Valued Kernels

We now introduce the background of matrix-valued positive definite kernels, which provides a most general framework for specifying vector-valued RKHS. We focus on the intuition and key ideas in our introduction, and refer the readers to Alvarez et al. (2012); Carmeli et al. (2006) for mathematical treatment.

Recall that a standard real-valued RKHS $\mathcal{H}_k$ consists of the closure of the linear span of its kernel function $k(\cdot, x)$ as shown in (1). Vector-valued RKHS can be defined in a similar way, but consist of the linear span of a *matrix-valued* kernel function:

$$f(x) = \sum_i K(x, x_i) c_i, \quad (6)$$

for any $\{c_i\} \subset \mathbb{R}^d$, where $K: \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}^{d \times d}$ is now a matrix-valued kernel function, and c_i are vector-valued weights. Similar to the scalar case, we can define an inner product structure $\langle f, g \rangle_{\mathcal{H}_K} = \sum_{ij} c_i^\top K(x_i, x_j) s_j$, where we assume $g = \sum_i K(x, x_i) s_i$ and hence a norm $\|f\|_{\mathcal{H}_K}^2 = \sum_{ij} c_i^\top K(x_i, x_j) c_j$. In order to make the inner product and norm well defined, the matrix-valued kernel K is required to be symmetric in that $K(x, x') = K(x', x)^\top$, and positive definite in that $\sum_{ij} c_i^\top K(x_i, x_j) c_j \geq 0$, $\{x_i\} \subset \mathbb{R}^d$ for any $\{c_i\} \subset \mathbb{R}^d$.

Mathematically, one can show that the closure of the set of functions in (6), equipped with the inner product defined above, defines a RKHS that we denote by $\mathcal{H}_K$. It is “reproducing” because it has the following reproducing property that generalizes the version for scalar-valued RKHS: for any $f \in \mathcal{H}_K$ and any $c \in \mathbb{R}^d$, we have

$$f(x)^\top c = \langle f(\cdot), K(\cdot, x) c \rangle_{\mathcal{H}_K}, \quad (7)$$

where it is necessary to introduce c because the result of the inner product of two functions must be a scalar. A simple example of matrix kernel is $K(x, x') = k(x, x') \mathbf{I}$, where $\mathbf{I}$ is the $d \times d$ identity matrix. Its related RKHS is $\mathcal{H}_K = \mathcal{H}_k \times \cdots \times \mathcal{H}_k = \mathcal{H}_k^d$, as used in the original SVGD.

3.2 SVGD with Matrix-Valued Kernels

It is now natural to leverage matrix-valued kernels to obtain a generalization of SVGD (see Algorithm 1). The idea is simple: we now optimize ϕ in the unit ball of a general vector-valued RKHS $\mathcal{H}_K$ with a matrix valued kernel $K(x, x')$.

$$\phi_K^* = \operatorname{argmax}_{\phi \in \mathcal{H}_K} \left\{ \mathbb{E}_{x \sim q} [\mathcal{P}^\top \phi(x)], \text{ s.t. } \|\phi\|_{\mathcal{H}_K} \leq 1 \right\}. \quad (8)$$

This yields a simple closed form solution similar to (5).

Theorem 1.—*Let $K(x, x')$ be a matrix-valued positive definite kernel that is continuously differentiable on x and x' , the optimal ϕ^* in (8) is*

$$\phi_K^*(\cdot) \propto \mathbb{E}_{x \sim q} [K(\cdot, x) \mathcal{P}] = \mathbb{E}_{x \sim q} [K(\cdot, x) \nabla_x \log p(x) + K(\cdot, x) \nabla_x], \quad (9)$$

Algorithm 1

Stein Variational Gradient Descent with Matrix-valued Kernels (Matrix SVGD)

Input: A (possibly unnormalized) differentiable density function $p(x)$ in $\mathbb{R}^d$. A matrix-valued positive definite kernel $K(x, x')$. Step size ϵ .

Goal: Find a set of particles $\{x_i\}_{i=1}^n$ to represent the distribution p .

Initialize a set of particles $\{x_i\}_{i=1}^n$, e.g., by drawing from some simple distribution.

repeat

$$x_i \leftarrow x_i + \frac{\epsilon}{n} \sum_{j=1}^n [K(x_i, x_j) \nabla_{x_j} \log p(x_j) + K(x_i, x_j) \nabla_{x_j}],$$

where $K(\cdot, x) \nabla_x$ is formally defined as the product of matrix $K(\cdot, x)$ and vector ∇_x . The ℓ -th element of $K(\cdot, x) \nabla_x$ is $(K(\cdot, x) \nabla_x)_\ell = \sum_{m=1}^d \nabla_{x^m} K_{\ell, m}(\cdot, x)$, see also (10).

until Convergence

where the Stein operator $\mathcal{P}$ and derivative operator ∇_x are again formally viewed as $\mathbb{R}^d$ -valued column vectors, and $K(\cdot, x) \mathcal{P}$ and $K(\cdot, x) \nabla_x$ are interpreted by the matrix multiplication rule. Therefore, $K(\cdot, x) \mathcal{P}$ is a $\mathbb{R}^d$ -valued column vector, whose ℓ -th element is defined by

$$(K(\cdot, x) \mathcal{P})_\ell = \sum_{m=1}^d (K_{\ell, m}(\cdot, x) \nabla_{x^m} \log p(x) + \nabla_{x^m} K_{\ell, m}(\cdot, x)), \quad (10)$$

where $K_{\ell, m}(x, x')$ denotes the (ℓ, m) -element of matrix $K(x, x')$ and x^m the m -th element of x .

Similar to the case of standard SVGD, recursively applying the optimal transform ϕ_K^* on the particles yields a general SVGD algorithm shown in Algorithm 1, which we call *matrix SVGD*.

Parallel to vanilla SVGD, the gradient of matrix SVGD in (9) consists of two parts that account for optimization and diversity, respectively: the first part is a weighted average of gradient $\nabla_x \log p(x)$ multiplied by a matrix-value kernel $K(\cdot, x)$ the other part consists of the gradient of the matrix-valued kernel K , which, like standard SVGD, serves as a repulsive force to keep the particles away from each other to reflect the uncertainty captured in distribution p .

Matrix SVGD includes various previous variants of SVGD as special cases. The vanilla SVGD corresponds to the case when $K(x, x') = k(x, x')\mathbf{I}$, with $\mathbf{I}$ as the $d \times d$ identity matrix; the gradient-free SVGD of Han & Liu (2018) can be treated as the case when $K(x, x') = k(x, x')w(x)w(x')\mathbf{I}$, where $w(x)$ is an importance weight function; the graphical SVGD of Wang et al. (2018); Zhuo et al. (2018) corresponds to a diagonal matrix-valued kernel: $K(x, x') = \text{diag}\left[\left\{k_\ell(x, x')\right\}_{\ell=1}^d\right]$, where each $k_\ell(x, x')$ is a “local” scalar-valued kernel function related to the ℓ -th coordinate x^ℓ of vector x .

3.3. Matrix-Valued Kernels and Change of Variables

It is well known that preconditioned gradient descent can be interpreted as applying standard gradient descent on a reparameterization of the variables. For example, let $y = Q^{1/2}x$, where Q is a positive definite matrix, then $\log p(x) = \log p(Q^{-1/2}y)$. Applying gradient descent on y and transform it back to the updates on x yields a preconditioned gradient update $x \leftarrow x + \epsilon Q^{-1} \nabla_x \log p(x)$.

We now extend this idea to SVGD, for which matrix-valued kernels show up naturally as a consequence of change of variables. This justifies the use of matrix-valued kernels and provides guidance on the practical choice of matrix-valued kernels. We start with a basic result of how matrix-valued kernels change under change of variables (see Paulsen & Raghupathi (2016)).

Lemma 2.—Assume $\mathcal{H}_0$ is an RKHS with a matrix kernel $K_0: \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}^{d \times d}$. Let $\mathcal{H}$ be the set of functions formed by

$$\phi(x) = M(x)\phi_0(t(x)), \quad \forall \phi_0 \in \mathcal{H}_0,$$

where $M: \mathbb{R}^d \rightarrow \mathbb{R}^{d \times d}$ is a fixed matrix-valued function and we assume $M(x)$ is an invertible matrix for all x , and $t: \mathbb{R}^d \rightarrow \mathbb{R}^d$ is a fixed continuously differentiable one-to-one transform on $\mathbb{R}^d$.

For $\forall \phi, \phi' \in \mathcal{H}$, we can identify an unique $\phi_0, \phi'_0 \in \mathcal{H}_0$ such that $\phi(x) = M(x)\phi_0(t(x))$ and $\phi'(x) = M(x)\phi'_0(t(x))$. Define the inner product on $\mathcal{H}$ via $\langle \phi, \phi' \rangle_{\mathcal{H}} = \langle \phi_0, \phi'_0 \rangle_{\mathcal{H}_0}$, then $\mathcal{H}$ is also a vector-valued RKHS, whose matrix-valued kernel is

$$K(x, x') = M(x)K_0(t(x), t(x'))M(x')^T.$$

We now present a key result, which characterizes the change of kernels when we apply invertible variable transforms on the SVGD trajectory.

Theorem 3.—i) Let p and q be two distributions and p_0, q_0 the distribution of $x_0 = t(x)$ when x is drawn from p, q , respectively, where t is a continuous differentiable one-to-one map on $\mathbb{R}^d$. Assume p is a continuous differentiable density with Stein operator $\mathcal{P}$, and $\mathcal{P}_0$ the Stein operator of p_0 . We have

$$\mathbb{E}_{x \sim q_0}[\mathcal{P}_0^T \phi_0(x)] = \mathbb{E}_{x \sim q}[\mathcal{P}^T \phi(x)], \quad \text{with} \quad \phi(x) := \nabla t(x)^{-1} \phi_0(t(x)), \quad (11)$$

where ∇t is the Jacobian matrix of t .

ii) Therefore, in the asymptotics of infinitesimal step size ($\epsilon \rightarrow 0^+$), running SVGD with kernel K_0 on p_0 is equivalent to running SVGD on p with kernel

$$K(x, x') = \nabla t(x)^{-1} K_0(t(x), t(x')) \nabla t(x')^{-T},$$

in the sense that the trajectory of these two SVGD can be mapped to each other by the one-to-one map t (and its inverse).

3.4. Practical Choice of Matrix-Valued Kernels

Theorem 3 suggests a conceptual procedure for constructing proper matrix kernels to incorporate desirable preconditioning information: one can construct a one-to-one map t so that the distribution p_0 of $x_0 = t(x)$ is an easy-to-sample distribution with a simpler kernel, which can be a standard scalar-valued kernel or a simple diagonal kernel. Practical choices of t often involve rotating x with either Hessian matrix or Fisher information, allowing us to incorporate these information into SVGD. In the sequel, we first illustrate this idea for simple Gaussian cases and then discuss practical approaches for non-Gaussian cases.

Constant Preconditioning Matrices—Consider the simple case when p is multivariate Gaussian, e.g., $\log p(x) = -\frac{1}{2}x^T Q x + \text{const}$, where Q is a positive definite matrix. In this case, the distribution p_0 of the transformed variable $t(x) = Q^{1/2}x$ is the standard Gaussian distribution that can be better approximated with a simpler kernel $K_0(x, x')$, which can be chosen to be the standard RBF kernel suggested in Liu & Wang (2016), the graphical kernel

suggested in Wang et al. (2018), or the linear kernels suggested in Liu & Wang (2018). Theorem 3 then suggests to use a kernel of form

$$K_Q(x, x') = Q^{-1/2} K_0(Q^{1/2}x, Q^{1/2}x') Q^{-1/2}, \quad (12)$$

in which Q is applied on both the input x and the output side. As an example, taking K_0 to be the scalar-valued Gaussian RBF kernel gives

$$K_Q(x, x') = Q^{-1} \exp\left(-\frac{1}{2h} \|x - x'\|_Q^2\right), \quad (13)$$

where $\|x - x'\|_Q^2 = (x - x')^\top Q (x - x')$ and h is a bandwidth parameter. Define

$K_{0,Q}(x, x') = K_0(Q^{1/2}x, Q^{1/2}x')$. One can show that the SVGD direction of the kernel in (12) equals

$$\phi_{K_Q}^*(\cdot) = Q^{-1} \mathbb{E}_{x \sim q} [\nabla \log p(x) K_{0,Q}(\cdot, x) + K_{0,Q}(\cdot, x) \nabla_x] = Q^{-1} \phi_{K_{0,Q}}^*(\cdot), \quad (14)$$

which is a linear transform of the SVGD direction of kernel $K_{0,Q}(x, x')$ with matrix Q^{-1} .

In practice, when p is non-Gaussian, we can construct Q by taking averaging over the particles. For example, denote by $H(x) = -\nabla_x^2 \log p(x)$ the negative Hessian matrix at x , we can construct Q by

$$Q = \sum_{i=1}^n H(x_i) / n, \quad (15)$$

where $\{x_i\}_{i=1}^n$ are the particles from the previous iteration. We may replace H with the Fisher information matrix to obtain a natural gradient like variant of SVGD.

Point-wise Preconditioning—A constant preconditioning matrix can not reflect different curvature or geometric information at different points. A simple heuristic to address this limitation is to replace the constant matrix Q with a point-wise matrix function $Q(x)$; this motivates a kernel of form

$$K(x, x') = Q^{-1/2}(x) K_0(Q^{1/2}(x)x, Q^{1/2}(x')x') Q^{-1/2}(x').$$

Unfortunately, this choice may yield expensive computation and difficult implementation in practice, because the SVGD update involves taking the gradient of the kernel $K(x, x')$, which would need to differentiate through matrix valued function $Q(x)$. When $Q(x)$ equals the Hessian matrix, for example, it involves taking the third order derivative of $\log p(x)$, yielding an unnatural algorithm.

Mixture Preconditioning—We instead propose a more practical approach to achieve point-wise preconditioning with a much simpler algorithm. The idea is to use a weighted combination of several constant preconditioning matrices. This involves leveraging a set of *anchor points* $\{z_\ell\}_{\ell=1}^m \subset \mathbb{R}^d$, each of which is associated with a preconditioning matrix $\mathbf{Q}_\ell = \mathbf{Q}(z_\ell)$ (e.g., their Hessian or Fisher information matrices). In practice, the anchor points $\{z_\ell\}_{\ell=1}^m$ can be conveniently set to be the same as the particles $\{x_i\}_{i=1}^n$. We then construct a kernel by

$$\mathbf{K}(\mathbf{x}, \mathbf{x}') = \sum_{\ell=1}^m \mathbf{K}_{\mathbf{Q}_\ell}(\mathbf{x}, \mathbf{x}') w_\ell(\mathbf{x}) w_\ell(\mathbf{x}'), \quad (16)$$

where $\mathbf{K}_{\mathbf{Q}_\ell}(\mathbf{x}, \mathbf{x}')$ is defined in (12) or (13), and $w_\ell(\mathbf{x})$ is a positive scalar-valued function that decides the contribution of kernel $\mathbf{K}_{\mathbf{Q}_\ell}$ at point $\mathbf{x}$. Here $w_\ell(\mathbf{x})$ should be viewed as a mixture probability, and hence should satisfy $\sum_{\ell=1}^m w_\ell(\mathbf{x}) = 1$ for all $\mathbf{x}$. In our empirical studies, we take $w_\ell(\mathbf{x})$ as the Gaussian mixture probability from the anchor points:

$$w_\ell(\mathbf{x}) = \frac{\mathcal{N}(\mathbf{x}; z_\ell, \mathbf{Q}_\ell^{-1})}{\sum_{\ell'=1}^m \mathcal{N}(\mathbf{x}; z_{\ell'}, \mathbf{Q}_{\ell'}^{-1})}, \quad \mathcal{N}(\mathbf{x}; z_\ell, \mathbf{Q}_\ell^{-1}) = \frac{1}{Z_\ell} \exp\left(-\frac{1}{2} \|\mathbf{x} - z_\ell\|_{\mathbf{Q}_\ell}^2\right), \quad (17)$$

where $Z_\ell = (2\pi)^{d/2} \det(\mathbf{Q}_\ell)^{-1/2}$. In this way, each point $\mathbf{x}$ is mostly influenced by the anchor point closest to it, allowing us to apply different preconditioning for different points. Importantly, the SVGD update direction related to the kernel in (16) has a simple and easy-to-implement form:

$$\phi_{\mathbf{K}}^*(\cdot) = \sum_{\ell=1}^m w_\ell(\cdot) \mathbb{E}_{\mathbf{x} \sim q} \left[\left(w_\ell(\mathbf{x}) \mathbf{K}_{\mathbf{Q}_\ell}(\cdot, \mathbf{x}) \right) \mathcal{P} \right] = \sum_{\ell=1}^m w_\ell(\cdot) \phi_{w_\ell \mathbf{K}_{\mathbf{Q}_\ell}}^*(\cdot), \quad (18)$$

which is a weighted sum of a number of SVGD directions with constant preconditioning matrices (but with an asymmetric kernel $w_\ell(\mathbf{x}) \mathbf{K}_{\mathbf{Q}_\ell}(\cdot, \mathbf{x})$).

A Remark on Stein Variational Newton (SVN)—Detommaso et al. (2018) provided a Newton-like variation of SVGD. It is motivated by an intractable functional Newton framework, and arrives a practical algorithm using a series of approximation steps. The update of SVN is

$$\mathbf{x}_i \leftarrow \mathbf{x}_i + \epsilon \tilde{\mathbf{H}}_i^{-1} \phi_{\mathbf{K}}^*(\mathbf{x}_i), \quad \forall i = 1, \dots, n, \quad (19)$$

where $\phi_{\mathbf{K}}^*(\cdot)$ is the standard SVGD gradient, and $\tilde{\mathbf{H}}_i$ is a Hessian like matrix associated with particle $\mathbf{x}_i$ defined by

$$\tilde{\mathbf{H}}_i = \mathbb{E}_{\mathbf{x} \sim q} \left[\mathbf{H}(\mathbf{x}) k(\mathbf{x}, \mathbf{x}_i)^2 + (\nabla_{\mathbf{x}_i} k(\mathbf{x}, \mathbf{x}_i)) \otimes 2 \right],$$

where $\mathbf{H}(\mathbf{x}) = -\nabla_{\mathbf{x}}^2 \log p(\mathbf{x})$ and $\mathbf{w} \otimes 2 = \mathbf{w} \mathbf{w}^\top$. Due to the approximation introduced in the derivation of SVN, it does not correspond to a standard functional gradient flow like SVGD (unless $\tilde{\mathbf{H}}_i = \mathbf{Q}$ for all i , in which case it reduces to using a constant preconditioning matrix on SVGD like (14)). SVN can be heuristically viewed as a “hard” variant of (18), which assigns each particle with its own preconditioning matrix with probability one, but the mathematical form do not match precisely. On the other hand, it is useful to note that the set of fixed points of SVN update (19) is the *identical* to that of the standard SVGD update with $\phi_k^*(\cdot)$, once all $\tilde{\mathbf{H}}_i$ are positive definite matrices. This is because at the fixed points of (19), we have $\tilde{\mathbf{H}}_i^{-1} \phi_k^*(\mathbf{x}_i) = 0$ for $\forall i = 1, \dots, n$, which is equivalent to $\phi_k^*(\mathbf{x}_i) = 0$, $\forall i$ when all the $\tilde{\mathbf{H}}_i$, $\forall i$ are positive definite. Therefore, SVN can be justified as an alternative fixed point iteration method to achieve the same set of fixed points as the standard SVGD.

4. Experiments

We demonstrate the effectiveness of our matrix SVGD on various practical tasks. We start with a toy example and then proceed to more challenging tasks that involve logistic regression, neural networks and recurrent neural networks. For our method, we take the preconditioning matrices to be either Hessian or Fisher information matrices, depending on the application. For large scale Fisher matrices in (recurrent) neural networks, we leverage the Kronecker-factored (KFAC) approximation by Martens & Grosse (2015); Martens et al. (2018) to enable efficient computation. We use RBF kernel for vanilla SVGD. The kernel $\mathbf{K}_0(\mathbf{x}, \mathbf{x}')$ in our matrix SVGD (see (12) and (13)) is also taken to be Gaussian RBF.

Following Liu & Wang (2016), we choose the bandwidth of the Gaussian RBF kernels using the standard median trick and use Adagrad (Duchi et al., 2011) for stepsize. Our code is available at https://github.com/dilinwang820/matrix_ssvgd.

The algorithms we test are summarized here:

Vanilla SVGD, using the code by Liu & Wang (2016);

Matrix-SVGD (average), using the constant preconditioning matrix kernel in (13), with $\mathbf{Q}$ to be either the average of the Hessian matrices or Fisher matrices of the particles (e.g., (15));

Matrix-SVGD (mixture), using the mixture preconditioning matrix kernel in (16), where we pick the anchor points to be particles themselves, that is, $\{z_\ell\}_{\ell=1}^m = \{\mathbf{x}_i\}_{i=1}^n$;

Stein variational Newton (SVN), based on the implementation of Detommaso et al. (2018);

Preconditioned Stochastic Langevin Dynamics (pSGLD), which is a variant of SGLD (Li et al., 2016), using a diagonal approximation of Fisher information as the preconditioned matrix.

4.1. Two-Dimensional Toy Examples

Settings—We start with illustrating our method using a Gaussian mixture toy model (Figure 1), with exact Hessian matrices for preconditioning. For fair comparison, we search the best learning rate for all algorithms exhaustively. We use 50 particles for all the cases. We use the same initialization for all methods with the same random seeds.

Results—Figure 1 show the results for 2D toy examples. Appendix B shows more visualization and results on more examples. We can see that methods with Hessian information generally converge faster than vanilla SVGD, and Matrix-SVGD (mixture) yields the best performance.

4.2. Bayesian Logistic Regression

Settings—We consider the Bayesian logistic regression model for binary classification. Let $\mathcal{D} = \{(\mathbf{x}_j, y_j)\}_{j=1}^N$ be a dataset with feature vector $\mathbf{x}_j$ and binary label $y_j \in \{0, 1\}$. The distribution of interest is

$$p(\boldsymbol{\theta}|\mathcal{D}) \propto p(\mathcal{D}|\boldsymbol{\theta})p(\boldsymbol{\theta}) \quad \text{with} \quad p(\mathcal{D}|\boldsymbol{\theta}) = \prod_{j=1}^N \left[y_j \sigma(\boldsymbol{\theta}^\top \mathbf{x}_j) + (1 - y_j) \sigma(-\boldsymbol{\theta}^\top \mathbf{x}_j) \right],$$

where $\sigma(z) := 1/(1 + \exp(-z))$, and $p_0(\boldsymbol{\theta})$ is the prior distribution, which we set to be standard normal $\mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, \mathbf{I})$. The goal is to approximate the posterior distribution $p(\boldsymbol{\theta}|\mathcal{D})$ with a set of particles $\{\boldsymbol{\theta}_i\}_{i=1}^n$, and then use it to predict the class labels for testing data points. We compare our methods with preconditioned stochastic gradient Langevin dynamics (pSGLD) (Li et al., 2016). Because pSGLD is a sequential algorithm, for fair comparison, we obtain the samples of pSGLD by running n parallel chains of pSGLD for estimation. The preconditioning matrix in both pSGLD and matrix SVGD is taken to be the Fisher information matrix.

We consider the binary *Covtype*² dataset with 581, 012 data points and 54 features. We partition the data into 70% for training, 10% for validation and 20% for testing. We use Adagrad optimizer with a mini-batch size of 256. We choose the best learning rate from [0.001, 0.005, 0.01, 0.05, 0.1, 0.5, 1.0] for each method on the validation set. For all the experiments and algorithms, we use $n = 20$ particles. Results are average over 20 random trials.

Results—Figure 2 (a) and (b) show the test accuracy and test log-likelihood of different algorithms. We can see that both Matrix-SVGD (average) and Matrix-SVGD (mixture) converge much faster than both vanilla SVGD and pSGLD, reaching an accuracy of 0.75 in less than 500 iterations.

²<https://www.csie.ntu.edu.tw/~cjlin/libsvmtools/datasets/binary.html>

4.3. Neural Network Regression

Settings—We apply our matrix SVGD on Bayesian neural network regression on UCI datasets. For all experiments, we use a two-layer neural network with 50 hidden units with ReLU activation functions. We assign isotropic Gaussian priors $\mathcal{N}(0, \alpha I)$ to the neural network weights. We set $\alpha = 0.5$. All datasets³ are randomly partitioned into 90% for training and 10% for testing. All results are averaged over 20 random trials, except for Protein and Year, on which 5 random trials are performed. We use $n = 10$ particles for all methods. We use Adam optimizer with a mini-batch size of 100; for large dataset such as *Year*, we set the mini-batch size to be 1000. We use the Fisher information matrix with Kronecker-factored (KFAC) approximation for preconditioning.

Results—Table 1 shows the performance in terms of the test RMSE and the test log-likelihood. We can see that both Matrix-SVGd (average) and Matrix-SVGd (mixture), which use second-order information, achieve better performance than vanilla SVGD. Matrix-SVGd (mixture) yields the best performance for both test RMSE and test log-likelihood in most cases. Figure 2 (c)-(d) show that both variants of Matrix-SVGd converge much faster than vanilla SVGD and pSGLD on the Protein dataset.

4.4 Sentence Classification With Recurrent Neural Networks (RNN)

Settings—We consider the sentence classification task on four datasets: MR (Pang & Lee, 2005), CR (Hu & Liu, 2004), SUBJ (Pang & Lee, 2004), and MPQA (Wiebe et al., 2005). We use a recurrent neural network (RNN) based model, $p(y|\mathbf{x}) = \text{softmax}(\mathbf{w}_y^\top \mathbf{h}_{RNN}(\mathbf{x}, \mathbf{v}))$, where $\mathbf{x}$ is the input sentence, y is a discrete-valued label of the sentence, and $\mathbf{w}_y$ is a weight coefficient related to label class y . And $\mathbf{h}_{RNN}(\mathbf{x}, \mathbf{v})$ is an RNN function with parameter $\mathbf{v}$ using a one-layer bidirectional GRU model (Cho et al., 2014) with 50 hidden units. We apply matrix SVGD to infer the posterior of $\mathbf{w} = \{\mathbf{w}_y: \forall y\}$, while updating the RNN weights $\mathbf{v}$ using typical stochastic gradient descent. In all experiments, we use the pre-processed text data provided in Gan et al. (2016). For all the datasets, we conduct 10-fold cross-validation for evaluation. We use $n = 10$ particles for all the methods. For training, we use a mini-batch size of 50 and run all the algorithms for 20 epochs. We use the Fisher information matrix for preconditioning.

Results—Table 2 shows the results of testing classification errors. We can see that Matrix-SVGd (mixture) generally performs the best among all algorithms.

5. Conclusion

We present a generalization of SVGD by leveraging general matrix-valued positive definite kernels, which allows us to flexibly incorporate various preconditioning matrices, including Hessian and Fisher information matrices, to improve exploration in the probability landscape. We test our practical algorithms on various practical tasks and demonstrate its efficiency compared to various existing methods.

³<https://archive.ics.uci.edu/ml/datasets.php>

Supplementary Material

Refer to Web version on PubMed Central for supplementary material.

Acknowledgement

This work is supported in part by NSF CRII 1830161 and NSF CAREER 1846421. We would like to acknowledge Google Cloud and Amazon Web Services (AWS) for their support.

A: Proof

Proof of Theorem 1

Let e_ℓ be the column vector with 1 in ℓ^{th} coordinate and 0 elsewhere. By the RKHS reproducing property (7) we have

$$\begin{aligned}
 \mathbb{E}_{\mathbf{x} \sim q} [\mathcal{P}^\top \boldsymbol{\phi}(\mathbf{x})] &= \mathbb{E}_{\mathbf{x} \sim q} [\nabla_{\mathbf{x}} \log p(\mathbf{x})^\top \boldsymbol{\phi}(\mathbf{x}) + \nabla_{\mathbf{x}}^\top \boldsymbol{\phi}(\mathbf{x})] \\
 &= \mathbb{E}_{\mathbf{x} \sim q} \left[\boldsymbol{\phi}(\mathbf{x})^\top \nabla_{\mathbf{x}} \log p(\mathbf{x}) + \sum_{\ell=1}^d \nabla_{\mathbf{x}}^\ell \boldsymbol{\phi}(\mathbf{x})^\top \mathbf{e}_\ell \right] \\
 &= \mathbb{E}_{\mathbf{x} \sim q} \left[\langle \boldsymbol{\phi}(\cdot), \mathbf{K}(\cdot, \mathbf{x}) \nabla_{\mathbf{x}} \log p(\mathbf{x}) \rangle_{\mathcal{H}_K} + \sum_{\ell=1}^d \nabla_{\mathbf{x}}^\ell \langle \boldsymbol{\phi}(\cdot), \mathbf{K}(\cdot, \mathbf{x}) \mathbf{e}_\ell \rangle_{\mathcal{H}_K} \right] \\
 &= \left\langle \boldsymbol{\phi}(\cdot), \mathbb{E}_{\mathbf{x} \sim q} \left[\mathbf{K}(\cdot, \mathbf{x}) \nabla_{\mathbf{x}} \log p(\mathbf{x}) + \sum_{\ell=1}^d \nabla_{\mathbf{x}}^\ell \mathbf{K}(\cdot, \mathbf{x}) \mathbf{e}_\ell \right] \right\rangle_{\mathcal{H}_K} \\
 &= \left\langle \boldsymbol{\phi}(\cdot), \mathbb{E}_{\mathbf{x} \sim q} [\mathbf{K}(\cdot, \mathbf{x}) \nabla_{\mathbf{x}} \log p(\mathbf{x}) + \mathbf{K}(\cdot, \mathbf{x}) \nabla_{\mathbf{x}}] \right\rangle_{\mathcal{H}_K} \\
 &= \left\langle \boldsymbol{\phi}(\cdot), \mathbb{E}_{\mathbf{x} \sim q} [\mathbf{K}(\cdot, \mathbf{x}) \mathcal{P}] \right\rangle_{\mathcal{H}_K},
 \end{aligned}$$

The optimization in (8) is hence

$$\max_{\boldsymbol{\phi} \in \mathcal{H}_K} \langle \boldsymbol{\phi}(\cdot), \mathbb{E}_{\mathbf{x} \sim q} [\mathbf{K}(\cdot, \mathbf{x}) \mathcal{P}] \rangle_{\mathcal{H}_K}, \quad \text{s.t.} \quad \|\boldsymbol{\phi}\|_{\mathcal{H}_K} \leq 1,$$

whose solution is $\boldsymbol{\phi}^*(\cdot) \propto \mathbb{E}_{\mathbf{x} \sim q} [\mathbf{K}(\cdot, \mathbf{x}) \mathcal{P}]$

Proof of Lemma 2

This is a basic result of RKHS, which can be found in classical textbooks such as Paulsen & Raghupathi (2016). The key idea is to show that $\mathbf{K}(\mathbf{x}, \mathbf{x}')$ satisfies the reproducing property for $\mathcal{H}$. Recall the reproducing property of $\mathcal{H}_0$:

$$\phi_0(x)^\top c = \langle \phi_0, K_0(\cdot, x)c \rangle_{\mathcal{H}_0}, \quad \forall c \in \mathbb{R}^d.$$

Taking $\phi(x) = M(x)\phi_0(t(x))$, we have

$$\begin{aligned} \phi(x)^\top c &= \langle \phi_0, K_0(\cdot, t(x))M(x)^\top c \rangle_{\mathcal{H}_0} \\ &= \langle \phi, M(\cdot)K_0(t(\cdot), t(x))M(x)^\top c \rangle_{\mathcal{H}} \\ &= \langle \phi, K(\cdot, x)c \rangle_{\mathcal{H}}, \end{aligned}$$

where the second step follows $\langle \phi, \phi' \rangle_{\mathcal{H}} = \langle \phi_0, \phi'_0 \rangle_{\mathcal{H}_0}$ with $\phi'_0(\cdot) = K_0(\cdot, t(x))M(x)^\top c$.

Proof of Theorem 3

Proof.

Note that KL divergence is invariant under invertible variable transforms, that is,

$$\text{KL}(q_{[\epsilon\phi]} \| p) = \text{KL}(q_{[\epsilon\phi]_0} \| p_0). \quad (20)$$

where p_0 denotes the distribution of $x_0 = t(x)$ when $x \sim p$, and $q^{[\epsilon\phi]_0}$ denotes the distribution of $x'_0 = t(x')$ when $x' \sim q_{[\epsilon\phi]}$. Recall that $q_{[\epsilon\phi]}$ is defined as the distribution of $x' = x + \epsilon\phi(x)$ when $x \sim q$.

Denote by t^{-1} the inverse map of t , that is, $t^{-1}(t(x)) = x$. We can see that $x'_0 \sim q_{[\epsilon\phi]_0}$ can be obtained by

$$\begin{aligned} x'_0 &= t(x') \quad // x' \sim q_{[\epsilon\phi]} \\ &= t(x + \epsilon\phi(x)) \quad // x \sim q \\ &= t(t^{-1}(x_0) + \epsilon\phi(t^{-1}(x_0))) \quad // x_0 \sim q_0 \\ &= x_0 + \epsilon \nabla t(t^{-1}(x_0)) \phi(t^{-1}(x_0)) + \mathcal{O}(\epsilon^2) \\ &= x_0 + \epsilon \phi_0(x_0) + \mathcal{O}(\epsilon^2), \end{aligned} \quad (21)$$

where we used the definition that $\phi(x) = \nabla t(x)^{-1} \phi_0(t(x))$ in (11), and $\mathcal{O}(\cdot)$ is the big-O notation.

From Theorem 3.1 of Liu & Wang (2016), we have

$$\frac{d}{d\epsilon} \text{KL}(q_{[\epsilon\phi]} \| p) \Big|_{\epsilon=0} = -\mathbb{E}_{q[\mathcal{D}^\top \phi]}.$$

Using Equation (21) and derivation similar to Theorem 3.1 of Liu & Wang (2016), we can show

$$\frac{d}{d\epsilon} \text{KL}(q_{[\epsilon\phi]0} \| p) \Big|_{\epsilon=0} = -\mathbb{E}_{q_0}[\mathcal{P}_0^\top \phi_0].$$

Combining these with (20) proves (11).

Following Lemma 2, when ϕ_0 is in $\mathcal{H}_0$ with kernel $K_0(x, x')$, ϕ is in $\mathcal{H}$ with kernel $K(x, x')$. Therefore, maximizing $\mathbb{E}_q[\mathcal{P}^\top \phi]$ in $\mathcal{H}$ is equivalent to $\mathbb{E}_{q_0}[\mathcal{P}_0^\top \phi_0]$ in $\mathcal{H}_0$. This suggests the trajectory of SVGD on p_0 with K_0 and that on p with K are equivalent. $\square$

B: Toy Examples

Figure 3 and Figure 4 show results of different algorithms on three 2D toy distributions: Star, Double banana and Sine. Detailed information of these distributions and more results are shown in Section B.1-B.3.

We can see from Figure 3–4 that both variants of matrix SVGD consistently outperform SVN and vanilla SVGD. We also find that Matrix SVGD(mixture) tends to outperform Matrix SVGD (average), which is expected since Matrix SVGD (average) uses a constant preconditioning matrix for all the particles, and can not capture different curvatures at different locations. Matrix SVGD (mixture) yields the best performance in general.

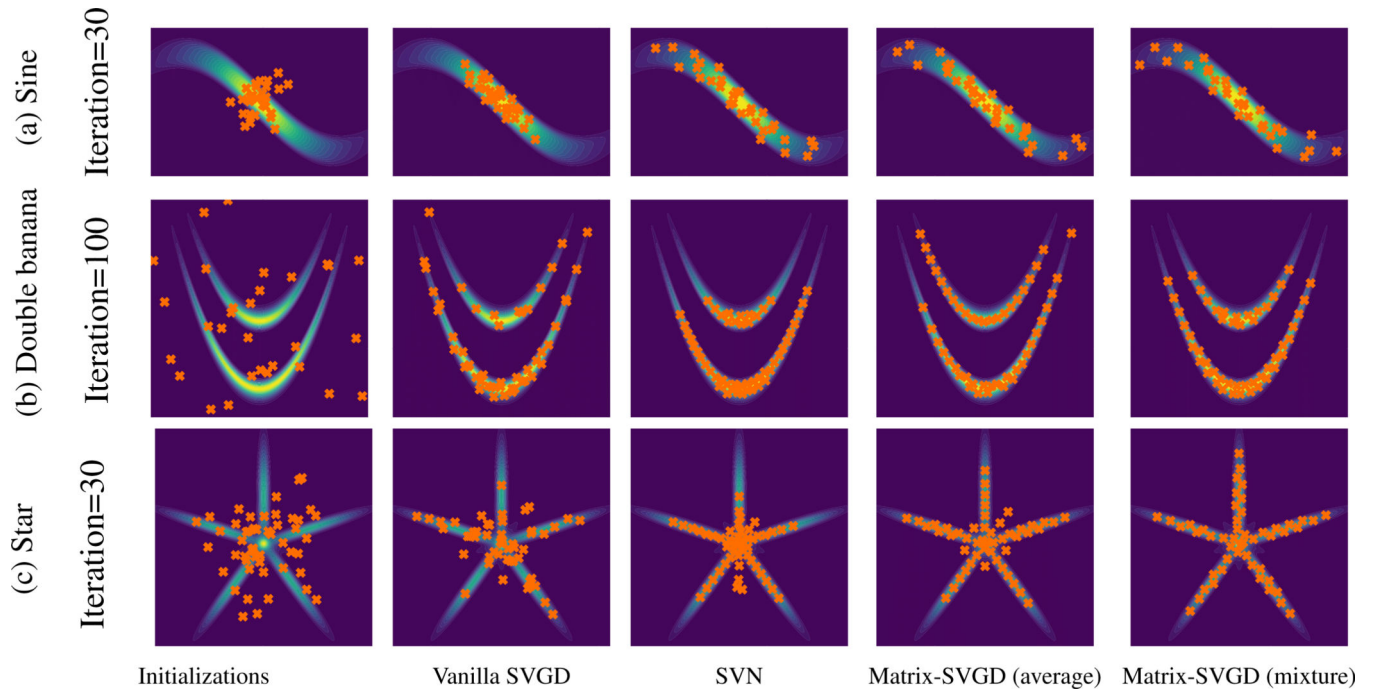
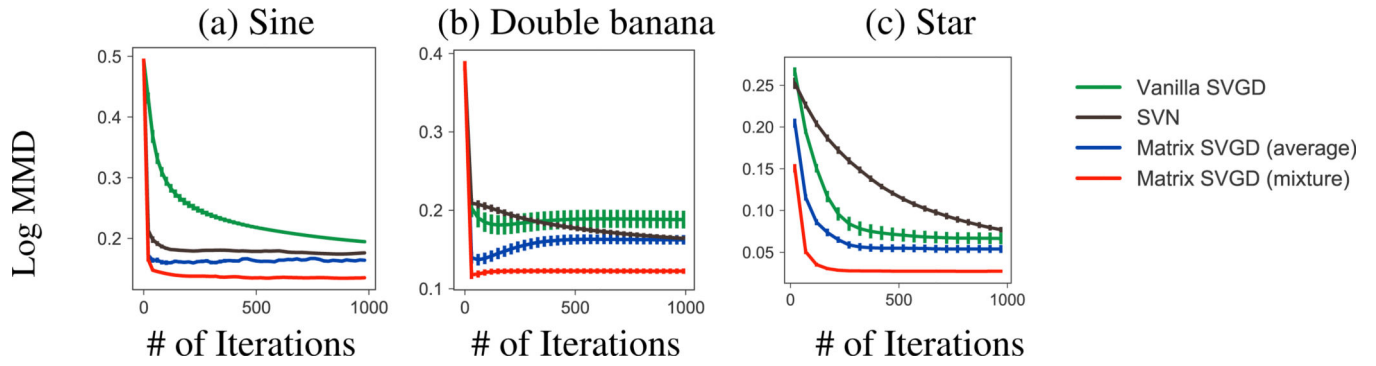


Figure 3:

The particles obtained by various methods at the 30/100/30-th iteration on three toy 2D distributions.

**Figure 4:**

The MMD vs. training iteration of different algorithms on the three toy distributions.

B.1 Sine

The density function of the “Sine” distribution is defined by

$$p(x_1, x_2) \propto \exp\left(\frac{-(x_2 + \sin(\alpha x_1))^2}{2\sigma_1} - \frac{x_1^2 + x_2^2}{2\sigma_2}\right),$$

where we choose $\sigma = 1$, $\sigma_1 = 0.003$, $\sigma_2 = 1$.

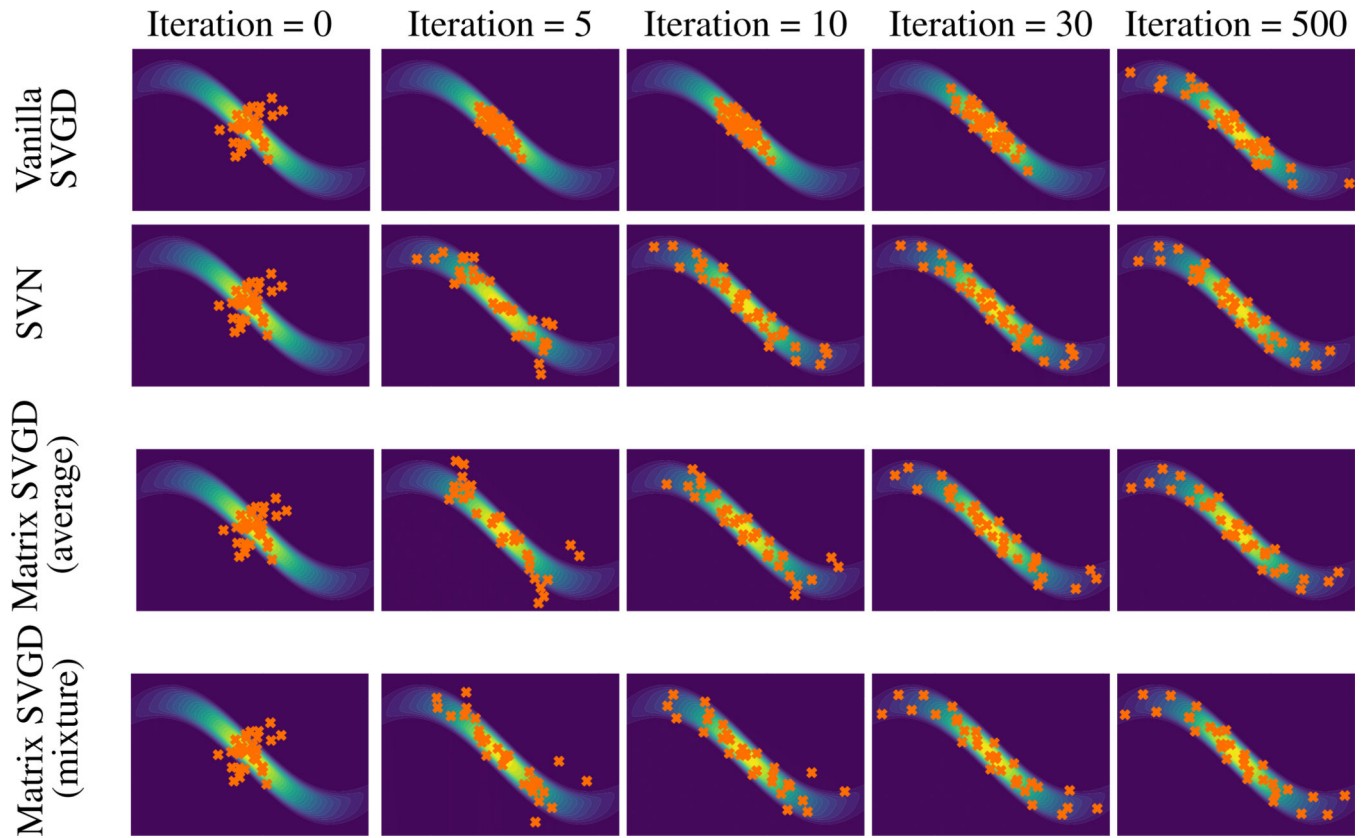


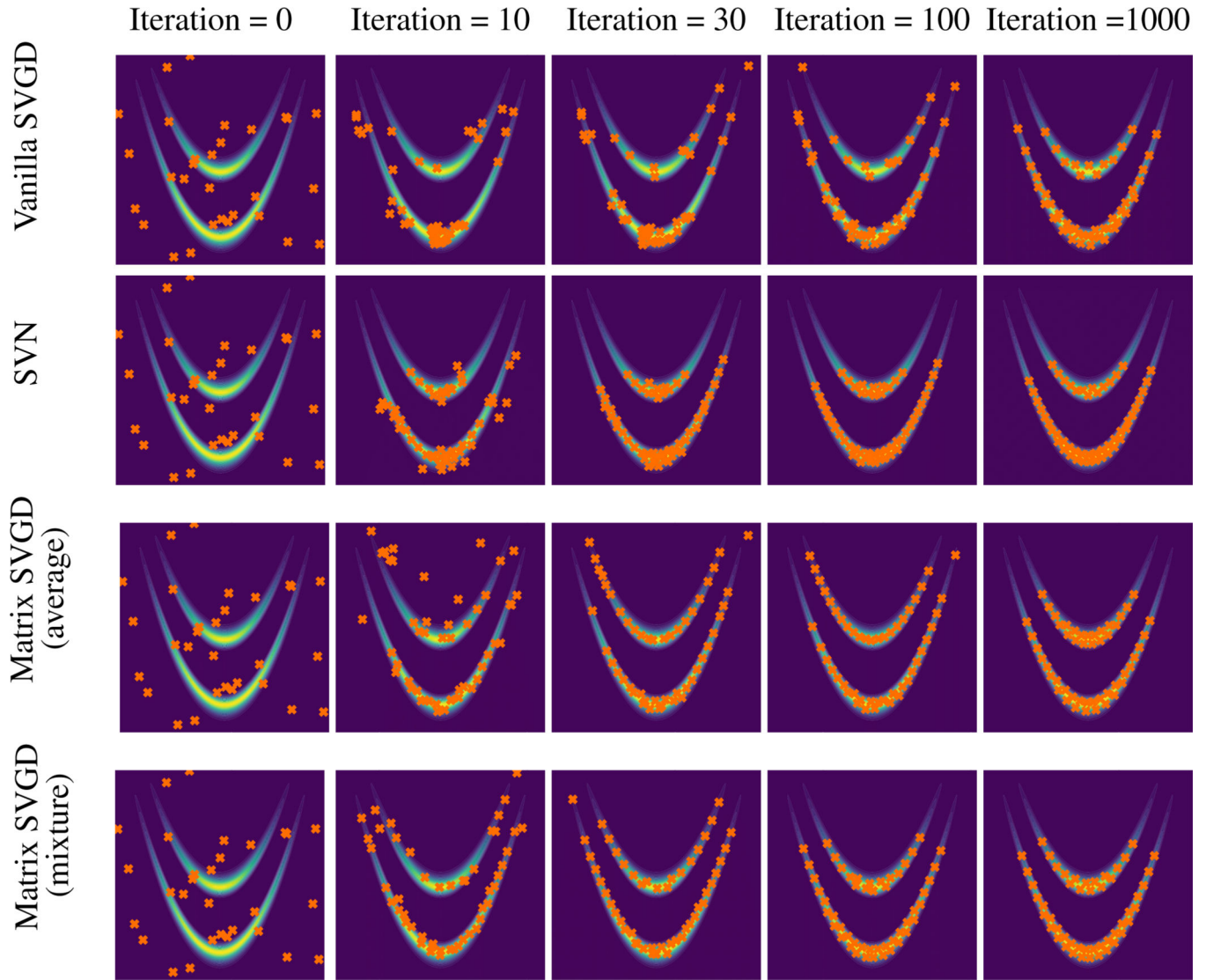
Figure 5:
The particles obtained by various methods on the toy Sine distribution.

B.2 Double Banana

We use the “double banana” distribution constructed in Detommaso et al. (2018), whose probability density function is

$$p(\mathbf{x}) \propto \exp\left(-\frac{\|\mathbf{x}\|_2^2}{2\sigma_1} - \frac{(y - F(\mathbf{x}))^2}{2\sigma_2}\right),$$

where $\mathbf{x} = [x_1, x_2] \in \mathbb{R}^2$ and $F(\mathbf{x}) = \log((1 - x_1)^2 + 100(x_2 - x_1^2)^2)$ and $y = \log(30)$, $\sigma_1 = 1.0$, $\sigma_2 = 0.09$.

**Figure 6:**

The particles obtained by various methods on the double banana distribution.

B.3 Star

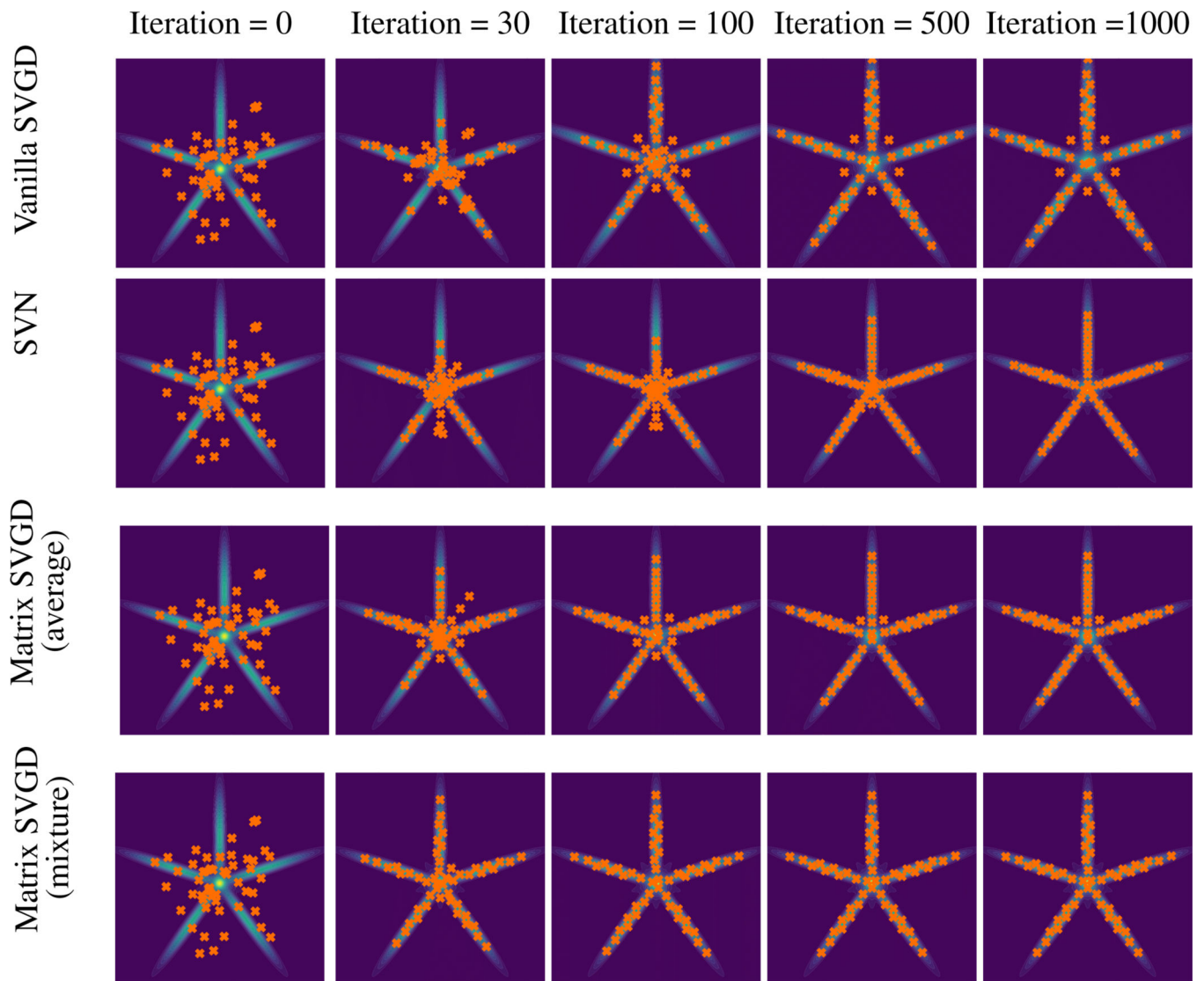
We construct the “star” distribution with a Gaussian mixture model, whose density function is

$$p(\mathbf{x}) = \frac{1}{K} \sum_{i=1}^K \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i),$$

with $\mathbf{x} \in \mathbb{R}^2$, $\boldsymbol{\mu}_1 = [0; 1.5]$, $\boldsymbol{\Sigma}_1 = \text{diag}\left(\left[1; \frac{1}{100}\right]\right)$, and the other means and covariance matrices are defined by rotating their previous mean and covariance matrix. To be precise,

$$\mu_{i+1} = U\mu_i, \quad \Sigma_{i+1} = U\Sigma_i U^\top, \quad U = \begin{bmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{bmatrix},$$

with angle $\theta = \frac{2\pi}{K}$. We set the number of component K to be 5.

**Figure 7:**

The particles obtained by various methods on the star-shaped distribution.

References

- Alvarez MA, Rosasco L, Lawrence ND, et al. Kernels for vector-valued functions: A review. *Foundations and Trends^{OR} in Machine Learning*, 4(3):195–266, 2012.
- Berlinet A and Thomas-Agnan C. Reproducing kernel Hilbert spaces in probability and statistics. Springer Science & Business Media, 2011.
- Blei DM, Kucukelbir A, and McAuliffe JD Variational inference: A review for statisticians. *Journal of the American Statistical Association*, 112(518):859–877, 2017.
- Carmeli C, De Vito E, and Toigo A. Vector valued reproducing kernel Hilbert spaces of integrable functions and Mercer theorem. *Analysis and Applications*, 4(04):377–408, 2006.
- Chen WY, Barp A, Briol F-X, Gorham J, Girolami M, Mackey L, Oates C, et al. Stein point markov chain monte carlo. *International Conference on Machine Learning (ICML)*, 2019.

- Cho K, Van Merriënboer B, Gulcehre C, Bahdanau D, Bougares F, Schwenk H, and Bengio Y. Learning phrase representations using RNN encoder-decoder for statistical machine translation. arXiv preprint arXiv:1406.1078, 2014.
- Detommaso G, Cui T, Marzouk Y, Spantini A, and Scheichl R. A Stein variational Newton method. In Advances in Neural Information Processing Systems, pp. 9187–9197, 2018.
- Duchi J, Hazan E, and Singer Y. Adaptive subgradient methods for online learning and stochastic optimization. Journal of Machine Learning Research, 12(7):2121–2159, 2011.
- Gan Z, Li C, Chen C, Pu Y, Su Q, and Carin L. Scalable Bayesian learning of recurrent neural networks for language modeling. ACL, 2016.
- Gretton A, Borgwardt KM, Rasch MJ, Schölkopf B, and Smola A. A kernel two-sample test. Journal of Machine Learning Research, 13(3):723–773, 2012.
- Haarnoja T, Tang H, Abbeel P, and Levine S. Reinforcement learning with deep energy-based policies. arXiv preprint arXiv:1702.08165, 2017.
- Han J and Liu Q. Stein variational gradient descent without gradient. In International Conference on Machine Learning, 2018.
- Hoffman MD and Gelman A. The No-U-turn sampler: adaptively setting path lengths in Hamiltonian Monte Carlo. Journal of Machine Learning Research, 15(1):1593–1623, 2014.
- Hu M and Liu B. Mining and summarizing customer reviews. In Proceedings of the tenth ACM SIGKDD international conference on Knowledge discovery and data mining, pp. 168–177. ACM, 2004.
- Kim T, Yoon J, Dia O, Kim S, Bengio Y, and Ahn S. Bayesian model-agnostic meta-learning. arXiv preprint arXiv:1806.03836, 2018.
- Li C, Chen C, Carlson DE, and Carin L. Preconditioned stochastic gradient Langevin dynamics for deep neural networks. In AAAI, volume 2, pp. 4, 2016.
- Liu C and Zhu J. Riemannian Stein variational gradient descent for Bayesian inference. In Thirty-Second AAAI Conference on Artificial Intelligence, 2018.
- Liu Q. Stein variational gradient descent as gradient flow. In Advances in neural information processing systems, pp. 3115–3123, 2017.
- Liu Q and Wang D. Stein variational gradient descent: A general purpose Bayesian inference algorithm. In Advances In Neural Information Processing Systems, pp. 2378–2386, 2016.
- Liu Q and Wang D. Stein variational gradient descent as moment matching. In Advances in Neural Information Processing Systems, pp. 8867–8876, 2018.
- Martens J and Grosse R. Optimizing neural networks with Kronecker-factored approximate curvature. In International conference on machine learning, pp. 2408–2417, 2015.
- Martens J, Ba J, and Johnson M. Kronecker-factored curvature approximations for recurrent neural networks. In ICLR, 2018.
- Neal RM et al. MCMC using Hamiltonian dynamics. Handbook of Markov Chain Monte Carlo, 2 (11): 2, 2011.
- Pang B and Lee L. A sentimental education: Sentiment analysis using subjectivity summarization based on minimum cuts. In Proceedings of the 42nd annual meeting on Association for Computational Linguistics, pp. 271 Association for Computational Linguistics, 2004.
- Pang B and Lee L. Seeing stars: Exploiting class relationships for sentiment categorization with respect to rating scales. In Proceedings of the 43rd annual meeting on association for computational linguistics, pp. 115–124. Association for Computational Linguistics, 2005.
- Paulsen VI and Raghupathi M. An introduction to the theory of reproducing kernel Hilbert spaces, volume 152 Cambridge University Press, 2016.
- Pu Y, Gan Z, Henao R, Li C, Han S, and Carin L. VAE learning via Stein variational gradient descent. In Advances in Neural Information Processing Systems, pp. 4236–4245, 2017.
- Wainwright MJ, Jordan MI, et al. Graphical models, exponential families, and variational inference. Foundations and Trends OR in Machine Learning, 1(1–2):1–305, 2008.
- Wang D and Liu Q. Learning to draw samples: With application to amortized MLE for generative adversarial learning. arXiv preprint arXiv:1611.01722, 2016.

- Wang D, Zeng Z, and Liu Q. Stein variational message passing for continuous graphical models. In International Conference on Machine Learning, pp. 5206–5214, 2018.
- Wiebe J, Wilson T, and Cardie C. Annotating expressions of opinions and emotions in language. Language resources and evaluation, 39(2–3):165–210, 2005.
- Zhuo J, Liu C, Shi J, Zhu J, Chen N, and Zhang B. Message passing Stein variational gradient descent. In International Conference on Machine Learning, pp. 6013–6022, 2018.

Author Manuscript

Author Manuscript

Author Manuscript

Author Manuscript

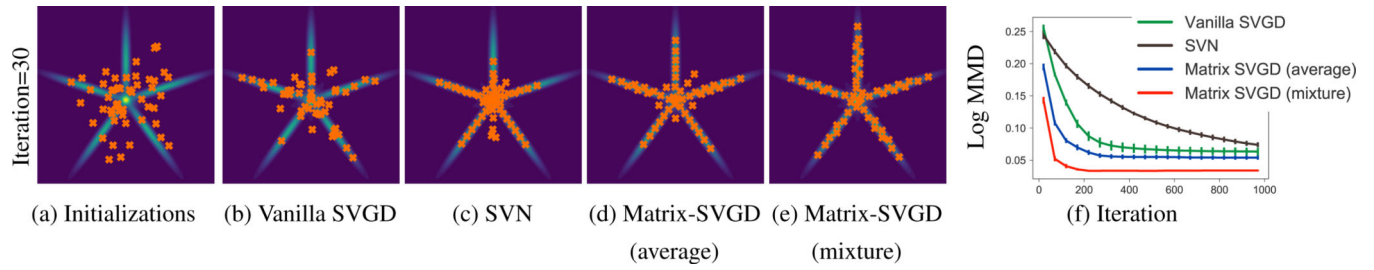


Figure 1:

Figure (a)-(e) show the particles obtained by various methods at the 30-th iteration. Figure (f) plots the log MMD (Gretton et al., 2012) vs. training iteration starting from the 10-th iteration. We use the standard RBF kernel for evaluating MMD.

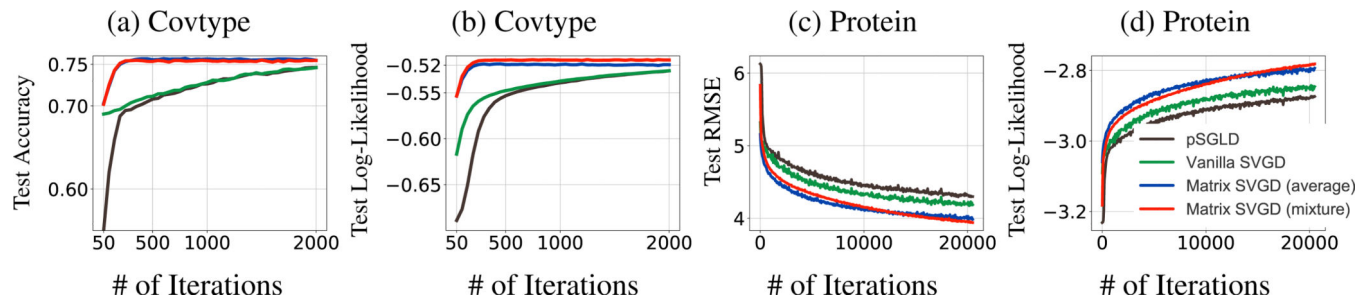


Figure 2:

(a)-(b) Results of Bayesian Logistic regression on the Covtype dataset. (c)-(d) Average test RMSE and log-likelihood vs. training batches on the Protein dataset for Bayesian neural regression.

Table 1:

Average test RMSE and log-likelihood in test data for UCI regression benchmarks.

Dataset	Test RMSE				Test Log-Likelihood			
	pSGLD	Vanilla SVGD	Matrix-SVGD (average)	Matrix-SVGD (mixture)	pSGLD	Vanilla SVGD	Matrix-SVGD (average)	Matrix-SVGD (mixture)
Boston	2.699±0.155	2.785±0.169	2.898±0.184	2.717±0.166	-2.847±0.182	-2.706±0.158	-2.669±0.141	-2.861±0.207
Concrete	5.053±0.124	5.027±0.116	4.869±0.124	4.721±0.111	-3.206±0.056	-3.064±0.034	-3.150±0.054	-3.207±0.071
Energy	0.985±0.024	0.889±0.024	0.795±0.025	0.868±0.025	-1.395±0.029	-1.315±0.020	-1.135±0.026	-1.249±0.036
Kin8nm	0.091±0.001	0.093±0.001	0.092±0.001	0.090±0.001	0.973±0.010	0.964±0.012	0.956±0.011	0.975±0.011
Naval	0.002±0.000	0.004±0.000	0.001±0.000	0.000±0.000	4.535±0.093	4.312±0.087	5.383±0.081	5.639±0.048
Combined	4.042±0.034	4.088±0.033	4.056±0.033	4.029±0.033	-2.821±0.009	-2.832±0.009	-2.824±0.009	-2.817±0.009
Wine	0.641±0.009	0.645±0.009	0.637±0.008	0.637±0.009	-0.984±0.016	-0.997±0.019	-0.980±0.016	-0.988±0.018
Protein	4.300±0.018	4.186±0.017	3.997±0.018	3.852±0.014	-2.874±0.004	-2.846±0.003	-2.796±0.004	-2.755±0.003
Year	8.630±0.007	8.686±0.010	8.637±0.005	8.594±0.009	-3.568±0.002	-3.577±0.002	-3.569±0.001	-3.561±0.002

Table 2:

Sentence classification errors measured with four benchmarks.

Method	MR	CR	SUBJ	MPQA
SGLD	20.52	18.65	7.66	11.24
pSGLD	19.75	17.50	6.99	10.80
Vanilla SVGD	19.73	18.07	6.67	10.58
Matrix-SVGD (average)	19.22	17.29	6.76	10.79
Matrix-SVGD (mixture)	19.09	17.13	6.59	10.71